Les namespaces en PHP

Exercices, solutions et quiz

Exercice 1 : Éviter les collisions de noms (Niveau Facile)

Vous développez une application de commerce en ligne. Vous devez créer deux classes qui portent le même nom : **Order**.

1. Créez un fichier contenant une classe **Order** située dans le namespace **App\Shop**. Cette classe doit posséder une méthode **getTotal()** qui retourne la chaîne "Total de la commande : 150€".

2. Dans le même script (ou un autre fichier), créez une classe **Order** située dans le namespace **App\Admin**. Cette classe doit posséder une méthode **getHistory()** qui retourne la chaîne "Historique de l'administration".

3. Créez un script d'exécution qui instancie ces deux classes et affiche le résultat de leurs méthodes respectives.

Solution - ShopOrder.php

```
<?php
namespace App\Shop;

class Order {
    public function getTotal() {
        return "Total de la commande : 150€";
    }
}
```

Solution - AdminOrder.php

```
<?php
namespace App\Admin;

class Order {
    public function getHistory() {
        return "Historique de l'administration";
    }
}
```

Solution - index.php (script d'exécution)

```
<?php
require_once 'ShopOrder.php';
require_once 'AdminOrder.php';

// Chemins absolus (Fully Qualified Names)
$shopOrder  = new \App\Shop\Order();
$adminOrder = new \App\Admin\Order();

echo $shopOrder->getTotal() . "\n";
echo $adminOrder->getHistory() . "\n";
```

Les trois fichiers doivent être dans le même dossier. Lancer avec : `php index.php`

Capture d'écran du code

[Placer ici la capture d'écran du code]

Capture d'écran de l'exécution

[Placer ici la capture d'écran du résultat (run)]

Exercice 2 : Importation et Alias (Niveau Moyen)

Dans cet exercice, vous allez manipuler le mot-clé **use** ainsi que les alias pour simplifier l'écriture de votre code.

1. Vous disposez d'une classe **Logger** dans le namespace **Core\Utils\Debug**.

2. Vous disposez d'une autre classe **Logger** issue d'une bibliothèque externe dans le namespace **Vendor\Log\External**.

3. Écrivez un fichier PHP dans le namespace **App\Controller** qui importe ces deux classes. Comme elles possèdent le même nom, vous devez obligatoirement attribuer l'alias **ExternalLogger** à la deuxième classe.

4. Instanciez les deux classes dans une fonction ou directement dans le script en utilisant leurs noms simplifiés.

Solution - fichier unique (à copier)

```
<?php
// Simulation des classes d'origine
namespace Core\Utils\Debug {
    class Logger {}
}

namespace Vendor\Log\External {
    class Logger {}
}

// ---- Fichier de travail ----
namespace App\Controller {

    use Core\Utils\Debug\Logger;
    use Vendor\Log\External\Logger as ExternalLogger; // Alias

    $localLogger = new Logger();         // Core\Utils\Debug\Logger
    $extLogger   = new ExternalLogger(); // Vendor\Log\External\Logger

    echo "Les deux loggers ont été instanciés avec succès !";
}
```

Remarque : quand un fichier contient plusieurs namespaces, PHP impose la syntaxe à accolades `namespace X { ... }` pour tous les namespaces du fichier. Mélanger accolades et déclarations simples provoque une erreur fatale.

Capture d'écran du code

[Placer ici la capture d'écran du code]

Capture d'écran de l'exécution

[Placer ici la capture d'écran du résultat (run)]

Exercice 3 : Le piège du Namespace Global (Niveau Avancé)

Le code ci-dessous génère une erreur fatale PHP lors de son exécution : **Fatal error: Class 'App\Services\DateTime' not found.**

1. Analysez le code et identifiez pourquoi PHP produit cette erreur.

2. Corrigez le code de deux manières différentes :

   - **Méthode A** : en modifiant directement la ligne de l'instanciation de l'objet.
   - **Méthode B** : en ajoutant une instruction d'importation au début du fichier.

```
<?php
namespace App\Services;

class DateFormat {
    public function showDate() {
        $date = new DateTime(); // Cette ligne provoque l'erreur
        return $date->format('d/m/Y');
    }
}

$manager = new DateFormat();
echo $manager->showDate();
```

Explication de l'erreur

Le fichier est déclaré dans le namespace **App\Services** : PHP cherche donc `DateTime` dans ce même espace (`App\Services\DateTime`). Or `DateTime` est une classe native qui se trouve dans le namespace global.

Solution - Méthode A (antislash global)

```
<?php
namespace App\Services;

class DateFormat {
    public function showDate() {
        $date = new \DateTime(); // Notez l'antislash initial
        return $date->format('d/m/Y');
    }
}

$manager = new DateFormat();
echo $manager->showDate();
```

Capture d'écran du code - Méthode A

[Placer ici la capture d'écran du code]

Capture d'écran de l'exécution - Méthode A

[Placer ici la capture d'écran du résultat (run)]

Solution - Méthode B (importation avec use)

```
<?php
namespace App\Services;

use DateTime; // Importation depuis le namespace global

class DateFormat {
    public function showDate() {
        $date = new DateTime(); // Fonctionne désormais
        return $date->format('d/m/Y');
    }
}

$manager = new DateFormat();
echo $manager->showDate();
```

Capture d'écran du code - Méthode B

[Placer ici la capture d'écran du code]

Capture d'écran de l'exécution - Méthode B

[Placer ici la capture d'écran du résultat (run)]

Quiz : Les namespaces en PHP

1. Où doit se situer obligatoirement l'instruction de déclaration d'un namespace dans un fichier PHP ?

A. N'importe où dans le fichier, du moment que c'est avant la première instanciation de classe.

B. Tout au début du fichier, immédiatement après la balise d'ouverture <?php (à l'exception des déclarations declare). (correct)

C. Juste avant la classe ou la fonction concernée par ce namespace.

D. À la toute fin du fichier PHP, juste avant la fermeture de la balise script.

La déclaration du namespace doit être la toute première instruction, juste après <?php (seul declare peut la précéder).

2. Que se passe-t-il si vous placez une instruction `echo "Bonjour";` avant la déclaration `namespace MonProjet;` dans un fichier PHP ?

A. Le texte s'affiche normalement et le namespace s'applique au reste du fichier sans problème.

B. PHP ignore silencieusement l'affichage et applique le namespace.

C. Une erreur fatale (Fatal Error) est déclenchée : "Namespace declaration statement has to be the very first statement". (correct)

D. Cela transforme le namespace en un sous-namespace implicite.

Toute sortie avant namespace provoque : Namespace declaration statement has to be the very first statement.

3. Si vous êtes à l'intérieur du namespace `MonProjet\Services`, comment accédez-vous explicitement à la classe native globale `DateTime` de PHP sans instruction `use` ?

A. En écrivant `\DateTime` avec une barre oblique inverse (backslash) au début. (correct)

B. En écrivant simplement `DateTime` car PHP regarde d'abord dans le namespace global.

C. En écrivant `global\DateTime`.

D. En écrivant `root:DateTime`.

L'antislash initial désigne le namespace global : \DateTime.

4. À l'intérieur du namespace `MonApp\Controllers`, vous écrivez `$db = new Connection();`. Comment PHP résout-il cette classe si aucun `use` n'est spécifié ?

A. Il cherche uniquement dans le namespace global, puis échoue.

B. Il cherche d'abord dans `MonApp\Controllers\Connection`. Si elle n'existe pas, il cherche dans le namespace global `Connection`. (correct)

C. Il génère immédiatement une erreur, car un `use` est obligatoire pour toute instanciation.

D. Il cherche d'abord dans tous les autres namespaces du projet de manière aléatoire.

Réponse du quiz : B. Mais attention, en vrai PHP, pour une classe, ce repli vers le global n'existe pas (cf. Exercice 3 : Class not found). Il ne vaut que pour les fonctions et constantes.

5. Quelle est la syntaxe correcte pour importer la classe `App\Entities\User` dans un fichier se trouvant dans un autre namespace ?

A. include App\Entities\User;

B. import App\Entities\User;

C. use App\Entities\User; (correct)

D. namespace App\Entities\User;

On importe une classe avec le mot-clé use.

6. Comment importer la classe `App\Http\Request` en lui attribuant l'alias `HttpRequest` pour éviter un conflit de nom dans votre code ?

A. use App\Http\Request rename HttpRequest;

B. use App\Http\Request as HttpRequest; (correct)

C. use HttpRequest = App\Http\Request;

D. use App\Http\Request(HttpRequest);

L'alias se définit avec le mot-clé as.

7. Vous devez importer simultanément deux classes nommées `Logger` de deux origines différentes : `App\Log\Logger` et `Vendor\Monolog\Logger`. Que devez-vous faire ?

A. Importer les deux normalement, PHP gérera le dernier import en écrasant le premier.

B. Utiliser le mot-clé `as` pour donner un alias distinct à au moins une des deux classes (ex: `use Vendor\Monolog\Logger as MonologLogger;`). (correct)

C. Les préfixer obligatoirement par un double point dans le code sans utiliser `use`.

D. Fusionner les deux namespaces en un seul bloc `use` combiné.

Deux classes de même nom exigent un alias (as) pour au moins l'une d'elles.

8. Quelle est la syntaxe correcte pour un import groupé (introduit en PHP 7) permettant d'importer les classes `Model`, `View` et `Controller` du namespace `App\Core` ?

A. use App\Core{Model, View, Controller}; (correct)

B. use App\Core(Model, View, Controller);

C. use App\Core\Model, View, Controller;

D. group use App\Core { Model, View, Controller };

Import groupé (PHP 7) : use App\Core{Model, View, Controller};

9. Comment déclare-t-on correctement un sous-namespace nommé `Database` qui se trouve sous le namespace principal `MonApplication` ?

A. namespace MonApplication.Database;

B. namespace MonApplication/Database;

C. subnamespace MonApplication\Database;

D. namespace MonApplication\Database; (correct)

L'antislash hiérarchise les namespaces : namespace MonApplication\Database;